

# Bloque 5 — Mini-especificaciones funcionales y técnicas

## Facturación y cobranza: F008 a F014

### 1. Propósito del bloque

Este bloque completa el módulo de facturación y pagos mediante las funciones de:

- Control de pagos con saldo disponible.
- Identificación de excedentes.
- Vinculación de pagos con movimientos bancarios.
- Seguimiento de acciones de cobranza.
- Registro de compromisos de pago.
- Alertas por vencimiento, promesas incumplidas y falta de respuesta.
- Antigüedad de saldos.
- Reportes y exportaciones.
- Pruebas integrales del módulo.

Al finalizar este bloque, el sistema podrá responder:

¿Qué pagos todavía tienen dinero sin aplicar?  
¿Qué pagos aparecen efectivamente en la cartola?  
¿Qué facturas están próximas a vencer?  
¿Qué facturas están vencidas?  
¿Qué gestiones de cobranza se han realizado?  
¿Qué compromisos de pago no se cumplieron?  
¿Cuánto debe cada cliente?  
¿Cuál es la antigüedad de esa deuda?

### 2. Resumen del bloque

| Código | Actividad                        | Horas | Recurso       | Cobertura actual  | Resultado                            |
|--------|----------------------------------|-------|---------------|-------------------|--------------------------------------|
| F008   | Excedentes y pagos sin aplicar   | 6 h   | Backend       | Parcial           | Saldo visible y regularizables       |
| F009   | Vinculación con cartola bancaria | 12 h  | Integraciones | Alta, con ajustes | Pago conciliado con movimientos      |
| F010   | Gestión de cobranza              | 6 h   | Frontend      | Parcial           | Historial de contactos y seguimiento |

| Código | Actividad                      | Horas | Recurso     | Cobertura actual | Resultado                          |
|--------|--------------------------------|-------|-------------|------------------|------------------------------------|
| F011   | Alertas de vencimiento         | 4 h   | Backend     | Parcial          | Alertas automáticas y gestionables |
| F012   | Antigüedad de saldos           | 6 h   | Frontend    | Sin modelo nuevo | Cartera agrupada por tramo         |
| F013   | Reportes y exportación         | 6 h   | Frontend    | Sin modelo nuevo | Informes filtrables y descargables |
| F014   | Pruebas de facturación y pagos | 12 h  | QA / Tester | Pendiente        | Validación integral del módulo     |

El bloque suma **52 horas**.

La Carta Gantt establece los siguientes resultados:

- F008: saldo no aplicado visible y regularizable.
- F009: un pago conciliable con uno o más movimientos bancarios.
- F010: llamadas, correos, respuestas y próximo seguimiento.
- F011: avisos por antigüedad, promesas y falta de respuesta.
- F012: tramos de vencimiento y cartera por cliente.
- F013: reportes de facturas pagadas, pendientes, parciales y vencidas.
- F014: pruebas de pagos múltiples, parciales, excedentes y anulaciones.

### 3. Secuencia funcional

```
F008 Pagos sin aplicar
→ F009 Vinculación con cartola
→ F010 Gestión de cobranza
→ F011 Alertas
→ F012 Antigüedad de saldos
→ F013 Reportes
→ F014 Pruebas integrales
```

El flujo de negocio completo será:

```
Factura emitida
→ pago registrado
→ pago distribuido entre facturas
→ pago vinculado con cartola
→ estado de cobranza actualizado
→ seguimiento de deuda pendiente
→ alertas
→ reportes
```

---

## 4. Tres saldos diferentes que deben distinguirse

Dentro del módulo existen tres conceptos monetarios diferentes.

### 4.1 Saldo de la factura

Saldo factura  
= total factura  
- aplicaciones de pago válidas

Indica cuánto continúa debiendo el cliente.

---

### 4.2 Saldo sin aplicar del pago

Saldo sin aplicar  
= monto del pago  
- aplicaciones activas a facturas

Indica cuánto dinero del pago todavía no ha sido distribuido entre facturas.

---

### 4.3 Saldo bancario pendiente del pago

Saldo bancario pendiente  
= monto del pago  
- aplicaciones bancarias activas

Indica cuánto del pago todavía no ha sido respaldado por movimientos de cartola.

Estos tres saldos pueden ser distintos.

Ejemplo:

|                           |                 |
|---------------------------|-----------------|
| Pago registrado:          | \$1.000.000 CLP |
| Aplicado a facturas:      | \$800.000 CLP   |
| Vinculado con cartola:    | \$600.000 CLP   |
|                           |                 |
| Saldo sin aplicar:        | \$200.000 CLP   |
| Saldo bancario pendiente: | \$400.000 CLP   |

El sistema no debe confundir distribución contable con conciliación bancaria.

---

## 5. Cobertura actual de modelos

`PagoCliente` ya calcula el monto distribuido y el saldo sin aplicar mediante sus relaciones con `AplicacionPagoFactura`.

`AplicacionMovimientoBancario` ya permite vincular un movimiento bancario con `PagoCliente`, por lo que F009 no requiere una nueva tabla puente. La relación existente admite varias aplicaciones bancarias hacia el mismo pago y permite que un movimiento se distribuya entre diferentes destinos.

`GestionCobranza` ya registra factura, fecha, tipo de contacto, contacto, mensaje, respuesta, próximo seguimiento y usuario responsable.

`AlertaCobranza` ya soporta alertas relacionadas con facturas vencidas, pagos sin factura, factoring y pagos directos sobre facturas cedidas. También contiene severidad, estado, fechas de apertura y resolución.

La cobertura es alta, pero necesita ampliaciones para:

- Diferenciar saldo distribuido y saldo conciliado.
- Registrar compromisos de pago.
- Detectar promesas vencidas.
- Detectar falta de respuesta.
- Evitar alertas duplicadas.
- Controlar anulaciones y regularizaciones.
- Generar reportes reproducibles a una fecha determinada.

---

## F008 — Excedentes y pagos sin aplicar

### 1. Objetivo

Identificar pagos que todavía conservan dinero sin distribuir entre facturas y permitir regularizarlos de forma controlada.

Ejemplo:

|                      |                 |
|----------------------|-----------------|
| Pago recibido:       | \$1.000.000 CLP |
| Aplicado a facturas: | \$750.000 CLP   |
| Saldo sin aplicar:   | \$250.000 CLP   |

Ese saldo puede corresponder a:

- Otra factura pendiente.
- Anticipo del cliente.
- Sobre pago.
- Pago duplicado.
- Error de identificación.

- Pago que debe devolverse.
- Pago de una factura todavía no registrada.
- Pago directo de una factura cedida.

---

## 2. Modelo principal

PagoCliente

Modelos relacionados:

AplicacionPagoFactura  
FacturaVenta  
DevolucionPagoCliente  
AlertaCobranza  
AplicacionMovimientoBancario

---

## 3. Ajuste conceptual al modelo

La propiedad actual `monto_aplicado` suma todas las aplicaciones que no estén anuladas.

Se recomienda distinguir:

```
@property
def monto_distribuido(self):
    """
    Total reservado o distribuido entre facturas,
    incluyendo aplicaciones pendientes o especiales.
    """
```

```
@property
def monto_aplicado_valido(self):
    """
    Total que efectivamente reduce deuda de facturas.
    Solo aplicaciones con estado APLICADA.
    """
```

```
@property
def saldo_sin_distribuir(self):
    return self.monto - self.monto_distribuido
```

Esto evita interpretar una aplicación `PAGO_ERRONEO_FACTORING` como pago válido de una factura, pero impide que el mismo dinero sea reutilizado mientras se regulariza.

---

## 4. Clasificación del saldo sin aplicar

Se propone agregar a `PagoCliente` :

```
class MotivoSaldo(models.TextChoices):
    PENDIENTE_IDENTIFICACION = (
        "PENDIENTE_IDENTIFICACION",
        "Pendiente de identificación",
    )
    ANTICIPO_CLIENTE = "ANTICIPO_CLIENTE", "Anticipo de cliente"
    SOBREPAGO = "SOBREPAGO", "Sobrepago"
    FACTURA_NO_REGISTRADA = (
        "FACTURA_NO_REGISTRADA",
        "Factura todavía no registrada",
    )
    PAGO_DUPLICADO = "PAGO_DUPLICADO", "Pago duplicado"
    PAGO_ERRONEO_FACTORING = (
        "PAGO_ERRONEO_FACTORING",
        "Pago erróneo de factura cedida",
    )
    POR_DEVOLVER = "POR_DEVOLVER", "Pendiente de devolución"
    OTRO = "OTRO", "Otro"
```

Campos:

```
motivo_saldo = models.CharField(
    max_length=40,
    choices=MotivoSaldo.choices,
    blank=True,
)
observacion_saldo = models.TextField(blank=True)
```

Estos campos solo serán obligatorios cuando exista saldo sin distribuir.

---

## 5. Vista de pagos sin aplicar

URL:

```
/administracion/pagos/sin-aplicar/
```

Filtros:

- Cliente.
- Fecha desde/hasta.

- Monto mínimo.
- Saldo mínimo.
- Motivo.
- Receptor.
- Con o sin conciliación bancaria.
- Antigüedad del pago.
- Con alerta abierta.

## 6. Columnas

| Columna          | Contenido                  |
|------------------|----------------------------|
| Pago             | Identificador              |
| Cliente          | Nombre y RUT               |
| Fecha            | Fecha del pago             |
| Monto            | Total                      |
| Distribuido      | Aplicaciones activas       |
| Sin aplicar      | Saldo                      |
| Conciliado banco | Monto vinculado a cartola  |
| Motivo           | Clasificación              |
| Días pendiente   | Antigüedad                 |
| Acción           | Aplicar, devolver, revisar |

## 7. Regularizaciones posibles

### Aplicar a factura existente

Saldo sin aplicar  
→ AplicacionPagoFactura

### Mantener como anticipo

El pago continúa con saldo disponible, pero queda clasificado y justificado.

### Devolver al cliente

PagoCliente  
→ DevolucionPagoCliente  
→ MovimientoBancario de egreso

## Relacionar con factura posteriormente registrada

La factura se crea y el pago se aplica manteniendo la fecha original.

## Marcar pago erróneo de factoring

Se activa el flujo que será desarrollado en X008-X010.

---

## 8. Estado del pago

Se recomienda mantener:

| Condición               | Estado     |
|-------------------------|------------|
| Anulado                 | ANULADO    |
| Sin distribución        | REGISTRADO |
| Distribución incompleta | PARCIAL    |
| Distribución completa   | APLICADO   |

El motivo del saldo será independiente del estado.

---

## 9. Alertas

Cuando un pago conserve saldo por más de un plazo definido, puede generarse:

PAGO\_SIN\_FACTURA

El modelo `AlertaCobranza` ya contempla este tipo.

La alerta se resolverá automáticamente cuando:

- El saldo llegue a cero.
- El pago sea anulado.
- Se complete una devolución.
- El saldo se clasifique como anticipo autorizado, según política.

---

## 10. URLs

```
/administracion/pagos/sin-aplicar/  
/administracion/pagos/<pk>/clasificar-saldo/
```



```
/administracion/pagos/<pk>/regularizar/  
/administracion/pagos/<pk>/solicitar-devolucion/
```

---

## 11. Vistas

```
PagosSinAplicarListView  
PagoClasificarSaldoView  
PagoRegularizarView  
PagoSolicitarDevolucionView
```

---

## 12. Servicio

```
services/pagos/excedentes.py
```

Funciones:

```
clasificar_saldo_pago()  
regularizar_saldo_pago()  
crear_devolucion_pago()  
recalcular_saldo_pago()  
resolver_alertas_pago()
```

---

## 13. Reglas de negocio

1. El saldo nunca puede ser negativo.
  2. Un pago anulado no tiene saldo utilizable.
  3. Un saldo no se elimina mediante una edición directa del monto.
  4. Todo excedente debe quedar visible.
  5. Todo saldo antiguo debe tener una clasificación.
  6. La devolución no puede superar el saldo disponible.
  7. Una devolución no libera el saldo hasta quedar confirmada.
  8. Un pago erróneo de factoring no puede aplicarse como pago normal.
  9. Una regularización debe dejar auditoría.
  10. Los saldos se calculan, no se ingresan manualmente.
- 

## 14. Pruebas

- Pago sin aplicaciones.
- Pago parcialmente distribuido.
- Pago completamente distribuido.

- Sobre pago.
- Anticipo.
- Devolución parcial.
- Devolución completa.
- Pago erróneo de factoring.
- Aplicación anulada que recupera saldo.
- Alerta por pago antiguo.
- Resolución automática de alerta.
- Concurrencia durante regularización.

---

## 15. Criterios de aceptación

- Todo pago muestra su saldo sin aplicar.
- Los saldos se filtran y clasifican.
- Los excedentes pueden regularizarse.
- No se reutiliza dinero reservado.
- Las devoluciones no superan el saldo.
- Los pagos antiguos generan seguimiento.
- Las regularizaciones quedan auditadas.

---

# F009 — Vinculación con cartola bancaria

## 1. Objetivo

Relacionar un pago registrado con uno o más movimientos bancarios que demuestren su recepción efectiva.

La Carta Gantt exige que un pago pueda conciliarse con uno o más movimientos.

Ejemplo:

|                        |                 |
|------------------------|-----------------|
| Pago registrado:       | \$1.000.000 CLP |
| Movimiento bancario A: | \$600.000 CLP   |
| Movimiento bancario B: | \$400.000 CLP   |

Resultado:

Pago bancariamente conciliado

## 2. Modelo utilizado

AplicacionMovimientoBancario

Con:

```
destino = PAGO_CLIENTE
pago_cliente = pago correspondiente
```

No se creará otra tabla de relación.

---

## 3. Dirección permitida

Un pago recibido por Extintores Real debe vincularse con movimientos:

INGRESO

Un pago informado como recibido directamente por una empresa de factoring no necesariamente aparecerá en la cuenta bancaria propia.

En ese caso:

- El pago puede existir sin conciliación en la cuenta de la empresa.
  - Su evidencia provendrá de información del factor.
  - No debe forzarse una cartola propia inexistente.
- 

## 4. Monto bancariamente conciliado

Se agregará a `PagoCliente`:

```
@property
def monto_conciliado_banco(self):
    total = self.aplicaciones_bancarias.filter(
        activa=True
    ).aggregate(t=Sum("monto"))["t"]
    return total or Decimal("0.00")
```

```
@property
def saldo_por_conciliar_banco(self):
    return self.monto - self.monto_conciliado_banco
```

```
@property
def estado_conciliacion_bancaria(self):
    if self.monto_conciliado_banco <= 0:
        return "NO_CONCILIADO"
    if self.monto_conciliado_banco < self.monto:
        return "PARCIAL"
    return "CONCILIADO"
```

No es obligatorio persistir este estado en una primera versión; puede calcularse mediante consultas anotadas.

## 5. Reglas de negocio

1. Solo se vinculan movimientos de ingreso.
2. El pago y el movimiento deben utilizar la misma moneda.
3. La suma bancaria no puede superar el pago.
4. La suma aplicada al movimiento no puede superar el movimiento.
5. Un movimiento puede cubrir varios pagos.
6. Un pago puede estar cubierto por varios movimientos.
7. Una vinculación puede ser parcial.
8. Un pago anulado no puede recibir aplicaciones.
9. Un movimiento excluido no puede utilizarse.
10. Un movimiento completamente conciliado no puede excederse.
11. Las aplicaciones no se eliminan físicamente.
12. Una reversa debe indicar motivo.
13. La fecha bancaria puede diferir de la fecha informada del pago.
14. Diferencias relevantes de fecha o monto deben generar advertencia.
15. La operación debe ser atómica.

## 6. Control del límite del pago

En B009 ya se definió que el sistema debe impedir que las aplicaciones bancarias acumuladas superen la operación destino.

Para `PagoCliente`:

```
Suma de aplicaciones bancarias activas
≤ monto del pago
```

Este control debe ejecutarse dentro de la transacción, bloqueando:

```
PagoCliente.objects.select_for_update()
MovimientoBancario.objects.select_for_update()
```

## 7. Sugerencias de movimientos

Desde un pago se propondrán movimientos candidatos según:

- Dirección de ingreso.
- Cuenta bancaria.
- Fecha cercana.
- Monto igual o cercano.
- Referencia coincidente.
- Contraparte.
- RUT.
- Saldo disponible del movimiento.
- Clasificación administrativa.

Puntuación conceptual:

|                                |     |
|--------------------------------|-----|
| Monto exacto                   | +40 |
| Referencia coincidente         | +25 |
| RUT coincidente                | +20 |
| Fecha dentro de tres días      | +10 |
| Nombre de contraparte parecido | +5  |

La puntuación solo sugiere; no concilia automáticamente en F009.

## 8. Interfaz desde el pago

URL:

```
/administracion/pagos/<pk>/conciliacion-bancaria/
```

Encabezado:

Pago #184  
Cliente: Cliente X  
Monto: \$1.000.000 CLP  
Conciliado: \$600.000 CLP  
Pendiente: \$400.000 CLP

Tabla de movimientos candidatos:

| Fecha      | Cuenta  | Descripción             | Monto disponible | Puntaje | Aplicar   |
|------------|---------|-------------------------|------------------|---------|-----------|
| 15-10-2026 | Banco A | Transferencia Cliente X | \$600.000        | 95      | \$600.000 |
| 16-10-2026 | Banco A | Abono REF-123           | \$400.000        | 80      | \$400.000 |

## 9. Interfaz desde el movimiento

También debe ser posible iniciar el proceso desde:

```
/administracion/movimientos/<pk>/aplicaciones/
```

Así se mantiene una navegación bidireccional:

```
Pago → movimientos  
Movimiento → pagos
```

---

## 10. Servicio

```
services/pagos/conciliacion_bancaria.py
```

Funciones:

```
buscar_movimientos_candidatos()  
calcular_puntaje_movimiento()  
conciliar_pago_con_movimientos()  
anular_vinculacion_bancaria()  
actualizar_estado_movimiento()
```

---

## 11. Auditoría

Registrar:

- Pago.
- Movimientos.
- Montos.
- Estado bancario anterior y nuevo.
- Estado de los movimientos.
- Usuario.
- Fecha.
- Motivo de reversa.

---

## 12. Pruebas

- Un pago con un movimiento.
- Un pago con dos movimientos.

- Un movimiento con dos pagos.
  - Conciliación parcial.
  - Conciliación completa.
  - Exceso sobre pago.
  - Exceso sobre movimiento.
  - Movimiento de egreso.
  - Movimiento excluido.
  - Moneda diferente.
  - Pago anulado.
  - Confirmaciones concurrentes.
  - Reversa.
  - Recalcular estados.
- 

## 13. Criterios de aceptación

- Un pago puede vincularse con varios movimientos.
  - Un movimiento puede cubrir varios pagos.
  - Los montos no se exceden.
  - La conciliación parcial es visible.
  - Los candidatos se sugieren sin aplicación automática.
  - Los estados bancarios se calculan correctamente.
  - La operación queda auditada.
  - La concurrencia no genera sobreconciliaciones.
- 

# F010 — Gestión de cobranza

## 1. Objetivo

Registrar cada acción realizada para cobrar una factura y planificar el siguiente seguimiento.

El modelo existente ya conserva llamadas, correos, WhatsApp, visitas, mensaje, respuesta, próximo contacto y usuario.

---

## 2. Modelo involucrado

GestionCobranza

Relacionado:

FacturaVenta  
ContactoCliente  
AlertaCobranza

---

### 3. Ajustes recomendados

Agregar resultados estructurados:

```
class Resultado(models.TextChoices):
    SIN_RESPUESTA = "SIN_RESPUESTA", "Sin respuesta"
    CONTACTADO = "CONTACTADO", "Contactado"
    PROMESA_PAGO = "PROMESA_PAGO", "Promesa de pago"
    PAGO_INFORMADO = "PAGO_INFORMADO", "Pago informado"
    RECLAMO_DOCUMENTO = (
        "RECLAMO_DOCUMENTO",
        "Reclamo sobre documento",
    )
    SOLICITA_PRORROGA = "SOLICITA_PRORROGA", "Solicita prórroga"
    DATOS_INCORRECTOS = "DATOS_INCORRECTOS", "Datos incorrectos"
    OTRO = "OTRO", "Otro"
```

Campos:

```
resultado = models.CharField(
    max_length=30,
    choices=Resultado.choices,
    blank=True,
)

contacto_cliente = models.ForeignKey(
    ContactoCliente,
    on_delete=models.SET_NULL,
    null=True,
    blank=True,
)

fecha_promesa_pago = models.DateField(
    null=True,
    blank=True,
)

monto_prometido = models.DecimalField(
    max_digits=14,
    decimal_places=2,
    null=True,
    blank=True,
)

completada = models.BooleanField(default=True)
```



## 4. Contacto histórico

Aunque se seleccione `ContactoCliente`, se mantendrá:

```
GestionCobranza.contacto
```

como snapshot del nombre utilizado.

Puede agregarse:

```
correo_contacto = models.EmailField(blank=True)
telefono_contacto = models.CharField(max_length=40, blank=True)
```

Así el historial no cambia si el contacto se modifica posteriormente.

## 5. Reglas de negocio

1. Solo se gestionan facturas con saldo.
2. Una factura anulada no recibe nuevas gestiones.
3. Una factura pagada puede mostrar historial, pero no nuevas acciones.
4. El tipo de contacto es obligatorio.
5. El usuario responsable se asigna automáticamente.
6. `PROMESA_PAGO` exige fecha.
7. El monto prometido no puede ser negativo.
8. La promesa puede ser parcial.
9. `SIN_RESPUESTA` exige próximo seguimiento.
10. La fecha del próximo contacto no puede ser anterior a la gestión.
11. Una gestión no se elimina físicamente.
12. Las correcciones deben quedar auditadas.

## 6. Pantalla de factura

Dentro del detalle de la factura:

```
Historial de cobranza
```

Tabla:

| Fecha      | Medio  | Contacto    | Resultado     | Respuesta | Próximo seguimiento |
|------------|--------|-------------|---------------|-----------|---------------------|
| 20-10-2026 | Correo | María Pérez | Sin respuesta | —         | 23-10-2026          |

| Fecha      | Medio   | Contacto    | Resultado       | Respuesta       | Próximo seguimiento |
|------------|---------|-------------|-----------------|-----------------|---------------------|
| 23-10-2026 | Llamada | María Pérez | Promesa de pago | Pagará el lunes | 26-10-2026          |

## 7. Bandeja de cobranza

URL:

```
/administracion/cobranza/
```

Filtros:

- Cliente.
- Factura.
- Estado.
- Tramo de antigüedad.
- Próximo seguimiento.
- Promesa pendiente.
- Sin respuesta.
- Responsable.
- Cedida a factoring.

## 8. Acciones

- Registrar gestión.
- Programar seguimiento.
- Registrar promesa.
- Marcar pago informado.
- Abrir pago nuevo.
- Consultar contactos.
- Resolver alerta relacionada.

## 9. URLs

```
/administracion/cobranza/
/administracion/facturas/<pk>/gestiones/
/administracion/facturas/<pk>/gestiones/nueva/
/administracion/gestiones-cobranza/<pk>/
/administracion/gestiones-cobranza/<pk>/editar/
```

## 10. Vistas

```
CobranzaPendienteListView  
GestionCobranzaCreateView  
GestionCobranzaDetailView  
GestionCobranzaUpdateView
```

---

## 11. Servicio

```
services/cobranza/gestiones.py
```

Funciones:

```
registrar_gestion()  
registrar_promesa_pago()  
programar_seguimiento()  
registrar_pago_informado()  
actualizar_alertas_cobranza()
```

---

## 12. Permisos

```
view_gestioncobranza  
add_gestioncobranza  
change_gestioncobranza  
gestionar_cartera
```

---

## 13. Auditoría

Registrar:

- Medio de contacto.
  - Persona contactada.
  - Resultado.
  - Promesa.
  - Próximo seguimiento.
  - Usuario.
  - Cambios posteriores.
-

## 14. Pruebas

- Registrar llamada.
  - Registrar correo.
  - Registrar sin respuesta.
  - Exigir próximo contacto.
  - Registrar promesa.
  - Exigir fecha de promesa.
  - Promesa parcial.
  - Factura pagada.
  - Factura anulada.
  - Contacto histórico.
  - Generación de alerta.
  - Permisos.
- 

## 15. Criterios de aceptación

- Cada acción de cobranza queda registrada.
  - Se conserva el contacto utilizado.
  - Se planifica el siguiente seguimiento.
  - Las promesas quedan estructuradas.
  - Se distinguen respuestas de falta de respuesta.
  - El historial se visualiza cronológicamente.
  - Las acciones quedan auditadas.
- 

# F011 — Alertas de vencimiento

## 1. Objetivo

Crear alertas operativas para que las facturas y compromisos pendientes no dependan únicamente de revisión manual.

La Carta Gantt solicita avisos por:

- Antigüedad.
  - Promesas.
  - Falta de respuesta.
- 

## 2. Modelo involucrado

AlertaCobranza

El modelo actual ya incluye:

- Tipo.
  - Severidad.
  - Estado.
  - Título.
  - Detalle.
  - Factura.
  - Operación de factoring.
  - Pago.
  - Fechas.
  - Usuario que resuelve.
- 

### 3. Tipos adicionales

Se recomienda incorporar:

```
FACTURA_POR_VENCER = (  
    "FACTURA_POR_VENCER",  
    "Factura próxima a vencer",  
)  
  
PROMESA_PAGO_VENCIDA = (  
    "PROMESA_PAGO_VENCIDA",  
    "Promesa de pago incumplida",  
)  
  
SEGUIMIENTO_VENCIDO = (  
    "SEGUIMIENTO_VENCIDO",  
    "Seguimiento de cobranza vencido",  
)  
  
SIN_RESPUESTA_COBRANZA = (  
    "SIN_RESPUESTA_COBRANZA",  
    "Cliente sin respuesta",  
)  
  
SALDO_PAGO_ANTIGUO = (  
    "SALDO_PAGO_ANTIGUO",  
    "Pago antiguo sin aplicar",  
)
```

---

### 4. Relación con gestión

Agregar:

```

gestion_cobranza = models.ForeignKey(
    GestionCobranza,
    on_delete=models.SET_NULL,
    null=True,
    blank=True,
    related_name="alertas",
)

```

Esto permite vincular una alerta de promesa o seguimiento con la gestión que la originó.

## 5. Prevención de duplicados

No debe generarse una alerta nueva cada vez que se ejecute el proceso diario.

Se propone:

```

clave_control = models.CharField(
    max_length=180,
    unique=True,
)

```

Ejemplos:

```

FACTURA_VENCIDA:factura:145
PROMESA_PAGO_VENCIDA:gestion:830
PAGO_SIN_FACTURA:pago:421

```

Cuando la alerta anterior esté resuelta y la condición reaparezca, podrá crearse una clave con versión o fecha de reapertura.

## 6. Severidades propuestas

| Condición                 | Severidad |
|---------------------------|-----------|
| Próxima a vencer          | Baja      |
| 1-15 días vencida         | Media     |
| 16-30 días vencida        | Alta      |
| Más de 30 días            | Crítica   |
| Promesa incumplida        | Alta      |
| Pago directo de factoring | Crítica   |

| Condición                    | Severidad |
|------------------------------|-----------|
| Pago sin identificar antiguo | Alta      |

Los rangos deben quedar configurados en un módulo de constantes o configuración, no dispersos en las vistas.

## 7. Proceso diario

Comando:

```
python manage.py generar_alertas_cobranza
```

Flujo:

```
Actualizar estados de facturas
→ buscar próximas a vencer
→ buscar vencidas
→ buscar promesas incumplidas
→ buscar seguimientos pendientes
→ buscar pagos sin aplicar
→ crear o actualizar alertas
→ resolver alertas cuya condición desapareció
```

## 8. Resolución automática

### Factura vencida

Resolver cuando:

- La factura se paga.
- Se anula.
- Se cede a factoring, según política.

### Promesa vencida

Resolver cuando:

- Se registra pago.
- Se crea una nueva promesa.
- El usuario descarta justificadamente la alerta.

## Pago sin factura

Resolver cuando:

- El saldo llega a cero.
  - Se clasifica como anticipo autorizado.
  - Se completa devolución.
  - El pago se anula.
- 

## 9. Bandeja de alertas

URL:

`/administracion/alertas-cobranza/`

Filtros:

- Tipo.
  - Severidad.
  - Estado.
  - Cliente.
  - Factura.
  - Pago.
  - Fecha.
  - Responsable.
- 

## 10. Acciones

- Abrir factura.
  - Registrar gestión.
  - Registrar pago.
  - Clasificar saldo.
  - Marcar en gestión.
  - Resolver.
  - Descartar con motivo.
- 

## 11. Reglas de negocio

1. No se crean alertas duplicadas.
2. Una alerta resuelta registra usuario y fecha.
3. Descartar exige motivo.
4. Las alertas críticas no se eliminan.
5. Las condiciones corregidas resuelven la alerta.
6. Reaparecer una condición debe quedar registrado.
7. Las alertas no modifican por sí solas saldos.



8. Las acciones continúan protegidas por permisos.

---

## 12. Pruebas

- Factura por vencer.
  - Factura vencida.
  - Cambio de severidad.
  - Promesa vencida.
  - Seguimiento vencido.
  - Pago sin aplicar.
  - Prevención de duplicados.
  - Resolución automática.
  - Descarte manual.
  - Reaparición.
  - Factura pagada.
  - Factura anulada.
  - Ejecución repetida del comando.
- 

## 13. Criterios de aceptación

- Las alertas se generan automáticamente.
  - No se duplican.
  - Se clasifican por severidad.
  - Se relacionan con su objeto.
  - Las condiciones corregidas resuelven alertas.
  - Las resoluciones quedan auditadas.
  - Existe una bandeja de trabajo.
- 

# F012 — Antigüedad de saldos

## 1. Objetivo

Mostrar la cartera pendiente agrupada según la cantidad de días transcurridos desde el vencimiento.

La Carta Gantt requiere tramos de vencimiento y cartera agrupada por cliente.

---

## 2. Tramos recomendados

Por vencer  
1-30 días  
31-60 días  
61-90 días

Más de 90 días  
Sin fecha de vencimiento

Los tramos deben calcularse respecto de una fecha de corte.

Ejemplo:

Fecha de corte: 31-10-2026

No siempre debe utilizarse la fecha actual, porque los reportes históricos requieren reproducibilidad.

### 3. Base del cálculo

Solo se considerará:

saldo pendiente válido

No el total original de la factura.

Ejemplo:

|                   |                 |
|-------------------|-----------------|
| Factura total:    | \$1.000.000 CLP |
| Pagado:           | \$600.000 CLP   |
| Saldo para aging: | \$400.000 CLP   |

### 4. Tratamiento de estados

#### Incluir

- Pendiente.
- Parcial.
- Vencida.

#### Excluir

- Pagada.
- Anulada.

#### Factoring

Las facturas cedidas deben mostrarse separadamente:

```
Cartera propia
Cartera cedida a factoring
Exposición con recurso
```

No conviene sumar automáticamente toda factura cedida como deuda ordinaria de cobranza propia.

---

## 5. Selector

```
selectors/aging_cobranza.py
```

Funciones:

```
obtener_antiguedad_saldos()
obtener_antiguedad_por_cliente()
obtener_totales_por_tramo()
obtener_detalle_tramo()
```

## 6. Consulta conceptual

Cada factura tendrá:

```
saldo pendiente
días vencida
tramo
```

La agrupación será:

```
Cliente
→ tramo
→ suma de saldos
```

## 7. Pantalla

URL:

```
/administracion/cobranza/antiguedad/
```

Encabezado:

- Fecha de corte.
- Cliente.
- Responsable de cobranza.
- Incluir factoring.
- Moneda.

---

## 8. Tabla resumida

| Cliente   | Por vencer | 1-30      | 31-60     | 61-90 | +90      | Total     |
|-----------|------------|-----------|-----------|-------|----------|-----------|
| Cliente A | \$500.000  | \$200.000 | \$0       | \$0   | \$0      | \$700.000 |
| Cliente B | \$0        | \$100.000 | \$300.000 | \$0   | \$50.000 | \$450.000 |

---

## 9. Detalle expandible

Al abrir un tramo:

| Factura | Emisión | Vencimiento | Días | Total | Pagado | Saldo |
|---------|---------|-------------|------|-------|--------|-------|
|---------|---------|-------------|------|-------|--------|-------|

---

## 10. Totales

Mostrar:

- Total cartera.
- Cartera vencida.
- Cartera por vencer.
- Cartera mayor a 90 días.
- Facturas cedidas.
- Exposición con recurso.
- Número de clientes con deuda.

---

## 11. Rendimiento

El cálculo debe realizarse mediante:

- Anotaciones.
- Agregaciones.
- Case  y  When .
- Consultas agrupadas.

No debe recorrerse toda la cartera ejecutando una consulta por factura.

---

## 12. Exportación

La misma consulta debe reutilizarse en F013.

No se debe reconstruir una lógica diferente para la exportación.

---

## 13. Pruebas

- Factura por vencer.
  - Factura exactamente en vencimiento.
  - 1 día vencida.
  - 30 días.
  - 31 días.
  - 60 días.
  - 61 días.
  - 90 días.
  - Más de 90.
  - Sin vencimiento.
  - Pago parcial.
  - Factura pagada.
  - Factura anulada.
  - Factoring.
  - Fecha de corte histórica.
  - Agrupación por cliente.
- 

## 14. Criterios de aceptación

- Los saldos se agrupan correctamente.
  - Se utiliza saldo pendiente.
  - Los límites de tramos son exactos.
  - La fecha de corte es configurable.
  - Factoring aparece diferenciado.
  - Se puede abrir el detalle.
  - La consulta es eficiente.
  - El reporte es reproducible.
- 

# F013 — Reportes y exportación

## 1. Objetivo

Generar informes operativos y financieros sobre facturas, pagos, cobranza y saldos.

La Carta Gantt requiere reportar facturas pagadas, pendientes, parciales y vencidas.

---

## **2. Reportes iniciales**

### **Reporte de facturas**

- Emitidas.
- Pendientes.
- Parciales.
- Pagadas.
- Vencidas.
- Cedidas.
- Anuladas.

### **Reporte de pagos**

- Registrados.
- Parcialmente aplicados.
- Aplicados.
- Anulados.
- Recibidos por la empresa.
- Informados al factoring.

### **Pagos sin aplicar**

- Cliente.
- Fecha.
- Monto.
- Saldo.
- Motivo.
- Antigüedad.

### **Conciliación bancaria de pagos**

- Pago.
- Monto.
- Monto conciliado.
- Saldo bancario.
- Movimientos relacionados.

### **Gestión de cobranza**

- Factura.
- Cliente.
- Última gestión.
- Resultado.
- Próximo contacto.
- Promesa.

## Antigüedad de saldos

- Resumen por cliente.
  - Resumen por tramo.
  - Detalle de facturas.
- 

## 3. Formatos

Primera fase:

XLSX  
CSV

Segunda fase:

PDF

XLSX será el formato principal para informes financieros detallados.

PDF se utilizará para informes de presentación o firma, no para grandes volúmenes de datos.

---

## 4. Arquitectura

```
Filtro
→ selector
→ conjunto de datos
→ exportador
→ archivo
```

No se duplicará la lógica de filtros entre:

- Pantalla.
  - Excel.
  - CSV.
  - PDF.
- 

## 5. Servicios

```
services/reportes/
├─ facturas.py
├─ pagos.py
├─ cobranza.py
```

```
|— aging.py
|— exportacion_xlsx.py
|— exportacion_csv.py
|— exportacion_pdf.py
```

---

## 6. Metadatos del reporte

Cada archivo debe incluir:

- Nombre.
- Fecha y hora de generación.
- Usuario.
- Fecha de corte.
- Filtros aplicados.
- Moneda.
- Cantidad de registros.
- Totales.

---

## 7. Nombres de archivo

Ejemplos:

```
facturas_vencidas_2026-10-31.xlsx
pagos_sin_aplicar_2026-10-31.xlsx
antigüedad_saldos_2026-10-31.xlsx
```

---

## 8. Reporte Excel

Hojas posibles:

```
Resumen
Detalle
Filtros
```

En antigüedad:

```
Resumen por cliente
Detalle de facturas
Factoring
```



## 9. Seguridad

1. El usuario debe tener permiso de exportación.
  2. Los datos respetan los mismos filtros y permisos de pantalla.
  3. Los archivos temporales deben eliminarse.
  4. Los reportes sensibles no deben quedar en URLs públicas.
  5. Las exportaciones deben auditarse.
  6. Los campos bancarios sensibles se enmascaran cuando corresponda.
  7. No se exportarán datos ajenos al alcance autorizado.
- 

## 10. Límites

Para exportaciones muy grandes:

- Limitar rango de fechas.
  - Informar cantidad máxima.
  - Utilizar escritura por streaming cuando el formato lo permita.
  - Evitar cargar todos los objetos completos en memoria.
  - Usar `values()` o iteradores.
- 

## 11. URLs

```
/administracion/reportes/facturas/  
/administracion/reportes/pagos/  
/administracion/reportes/pagos-sin-aplicar/  
/administracion/reportes/conciliacion-pagos/  
/administracion/reportes/cobranza/  
/administracion/reportes/antiguedad/
```

Cada reporte tendrá acciones:

```
[Consultar]  
[Exportar XLSX]  
[Exportar CSV]
```

---

## 12. Permisos

```
view_reportes_facturacion  
exportar_reportes_facturacion  
exportar_datos_sensibles
```

---

## 13. Auditoría

Registrar:

- Usuario.
  - Reporte.
  - Formato.
  - Filtros.
  - Número de registros.
  - Fecha.
  - Nombre del archivo.
- 

## 14. Pruebas

- Exportar facturas.
  - Exportar pagos.
  - Exportar aging.
  - Totales iguales a pantalla.
  - Filtros iguales a pantalla.
  - Archivo Excel válido.
  - Archivo CSV válido.
  - Fechas y montos correctos.
  - Caracteres especiales.
  - Archivo vacío.
  - Permiso denegado.
  - Volumen alto.
  - Auditoría.
- 

## 15. Criterios de aceptación

- Existen reportes principales.
  - Pantalla y exportación entregan los mismos datos.
  - Los totales coinciden.
  - Los filtros quedan documentados.
  - Los archivos se abren correctamente.
  - Las exportaciones respetan permisos.
  - Cada descarga queda auditada.
- 

# F014 — Pruebas de facturación y pagos

## 1. Objetivo

Validar el módulo completo antes de considerarlo apto para uso operativo.

La Carta Gantt asigna 12 horas y exige probar:

- Pagos múltiples.
- Pagos parciales.
- Excedentes.
- Anulaciones.

La prueba debe incluir además:

- Facturas.
  - Estados.
  - Banco.
  - Cobranza.
  - Alertas.
  - Reportes.
  - Permisos.
  - Concurrencia.
- 

## 2. Niveles de prueba

### Unitarias

Validan:

- Cálculos.
- Estados.
- Validaciones.
- Tramos.
- Fingerprints.
- Severidades.

### Servicios

Validan:

- Registro de factura.
- Registro de pago.
- Aplicación.
- Conciliación.
- Anulación.
- Regularización.
- Alertas.

### Integración

Validan relaciones entre:

Cliente  
Factura

Pago  
Aplicación  
Movimiento bancario  
Gestión  
Alerta

## Vistas

Validan:

- Formularios.
- URLs.
- Redirecciones.
- Mensajes.
- Filtros.
- Permisos.

## End-to-end del módulo

Validan flujos completos desde factura hasta conciliación y cobranza.

## 3. Escenarios mínimos

### Escenario 1 — Factura pagada completamente

Crear factura \$1.000.000  
→ registrar pago \$1.000.000  
→ aplicar a factura  
→ vincular movimiento bancario

Resultado:

Factura PAGADA  
Pago APLICADO  
Pago bancariamente CONCILIADO  
Movimiento CONCILIADO

### Escenario 2 — Pago parcial

Factura \$1.000.000  
→ pago \$400.000

Resultado:

Factura PARCIAL  
Saldo \$600.000  
Pago APLICADO

---

### Escenario 3 — Un pago para varias facturas

Pago \$900.000  
→ Factura A \$500.000  
→ Factura B \$400.000

Resultado:

Pago APLICADO  
Ambas facturas PAGADAS

---

### Escenario 4 — Varios pagos para una factura

Factura \$1.000.000  
→ pago A \$300.000  
→ pago B \$700.000

Resultado:

Factura PAGADA  
Historial conserva ambos pagos

---

### Escenario 5 — Excedente

Pago \$1.200.000  
→ factura \$1.000.000

Resultado:

Pago PARCIAL  
Saldo sin aplicar \$200.000  
Motivo requerido

---

## Escenario 6 — Pago sin factura

Registrar pago  
→ no aplicar durante plazo definido

Resultado:

Alerta PAGO\_SIN\_FACTURA

## Escenario 7 — Conciliación parcial

Pago \$1.000.000  
→ movimiento bancario \$600.000

Resultado:

Saldo bancario pendiente \$400.000

## Escenario 8 — Conciliación con varios movimientos

Pago \$1.000.000  
→ movimiento A \$600.000  
→ movimiento B \$400.000

Resultado:

Pago bancariamente conciliado

## Escenario 9 — Factura vencida

Factura sin pago  
→ superar fecha de vencimiento  
→ ejecutar comando diario

Resultado:

Estado VENCIDA  
Alerta abierta  
Aging correcto

---

## Escenario 10 — Promesa incumplida

Registrar gestión  
→ promesa de pago  
→ superar fecha  
→ no registrar pago

Resultado:

Alerta PROMESA\_PAGO\_VENCIDA

---

## Escenario 11 — Anulación de aplicación

Factura parcial  
→ anular aplicación

Resultado:

Factura recupera saldo  
Pago recupera saldo  
Historial conserva aplicación

---

## Escenario 12 — Pago directo de factura cedida

Factura cedida  
→ cliente paga a Extintores Real

Resultado:

Aplicación PAGO\_ERRONEO\_FACTORIZING  
Factura continúa CEDIDA\_FACTORIZING  
Alerta crítica

---

## 4. Pruebas de límites

- Monto cero.
  - Monto negativo.
  - Centavos o decimales.
  - Total extremadamente alto.
  - Fecha de vencimiento igual a emisión.
  - Fecha anterior.
  - Número duplicado.
  - Factura de otro cliente.
  - Pago anulado.
  - Factura anulada.
  - Pago excedido.
  - Factura sobrepagada.
  - Movimiento excedido.
  - Pago bancariamente sobreconciliado.
  - Aplicación duplicada.
  - Archivo vacío.
  - Reporte sin resultados.
- 

## 5. Pruebas de concurrencia

### Aplicación de pago

Dos usuarios intentan utilizar el mismo saldo.

Resultado:

```
Una operación se confirma  
La otra recibe error controlado  
No existe sobreaplicación
```

### Conciliación bancaria

Dos usuarios intentan utilizar el mismo saldo del movimiento.

Resultado:

```
No se excede el movimiento
```

### Anulación y aplicación simultáneas

Resultado:



Consistencia transaccional  
Sin estados intermedios inválidos

## 6. Pruebas de permisos

| Acción            | Usuario autorizado    | Usuario sin permiso |
|-------------------|-----------------------|---------------------|
| Crear factura     | Permitido             | Denegado            |
| Registrar pago    | Permitido             | Denegado            |
| Aplicar pago      | Permitido             | Denegado            |
| Anular aplicación | Permitido con permiso | Denegado            |
| Conciliar         | Permitido             | Denegado            |
| Exportar          | Permitido             | Denegado            |
| Ver auditoría     | Auditor               | Denegado            |

## 7. Pruebas de rendimiento

Objetivos iniciales:

- Listado paginado sin consultas N+1.
- Aging con consulta agregada.
- Exportación sin cargar objetos innecesarios.
- Generación de alertas por lotes.
- Aplicaciones sin consultas repetitivas.

Se recomienda utilizar pruebas de conteo de consultas en los listados principales.

## 8. Datos de prueba

Crear fábricas:

```
ClienteFactory  
ContactoClienteFactory  
FacturaVentaFactory  
PagoClienteFactory  
AplicacionPagoFacturaFactory  
MovimientoBancarioFactory  
AplicacionMovimientoBancarioFactory  
GestionCobranzaFactory
```

```
AlertaCobranzaFactory
OperacionFactoringFactory
```

## 9. Estructura de tests

```
tests/facturacion/
├─ test_clientes.py
├─ test_facturas.py
├─ test_estados_cobranza.py
├─ test_pagos.py
├─ test_aplicaciones.py
├─ test_excedentes.py
├─ test_conciliacion_bancaria.py
├─ test_gestion_cobranza.py
├─ test_alertas.py
├─ test_aging.py
├─ test_reportes.py
├─ test_permisos.py
└─ test_concurrencia.py
```

## 10. Criterios de aceptación

F014 se considerará terminada cuando:

- Todos los escenarios críticos pasan.
- No se superan pagos ni facturas.
- No se superan movimientos bancarios.
- Las anulaciones restituyen saldos.
- Los estados se actualizan.
- Las alertas no se duplican.
- El aging coincide con los saldos.
- Los reportes coinciden con pantalla.
- Los permisos están cubiertos.
- Las transacciones revierten completamente ante error.
- No existen consultas N+1 críticas.
- Las pruebas pueden ejecutarse repetidamente.

## 6. Estructura de archivos del bloque

```
administracion/
├─ models/
│   └─ facturacion.py
```

```
|
|
|— forms/
|   |— excedentes.py
|   |— conciliacion_pagos.py
|   |— gestion_cobranza.py
|   |— reportes_facturacion.py
|
|— views/
|   |— excedentes.py
|   |— conciliacion_pagos.py
|   |— gestion_cobranza.py
|   |— alertas_cobranza.py
|   |— aging.py
|   |— reportes_facturacion.py
|
|— services/
|   |— pagos/
|   |   |— excedentes.py
|   |   |— conciliacion_bancaria.py
|   |— cobranza/
|   |   |— gestiones.py
|   |   |— alertas.py
|   |   |— estados.py
|   |— reportes/
|   |   |— facturas.py
|   |   |— pagos.py
|   |   |— cobranza.py
|   |   |— aging.py
|   |   |— exportacion_xlsx.py
|   |   |— exportacion_csv.py
|   |   |— exportacion_pdf.py
|
|— selectors/
|   |— pagos_sin_aplicar.py
|   |— conciliacion_pagos.py
|   |— cobranza.py
|   |— alertas.py
|   |— aging_cobranza.py
|
|— management/commands/
|   |— actualizar_estados_cobranza.py
|   |— generar_alertas_cobranza.py
|
|— templates/administracion/
|   |— pagos/
|   |— cobranza/
|   |— alertas/
|   |— reportes/
|
|— tests/facturacion/
```

---

## 7. Migraciones previstas

### Migración 1 — Pago

Agregar:

```
motivo_saldo
observacion_saldo
fingerprint, si fue aprobado
```

### Migración 2 — Gestión de cobranza

Agregar:

```
resultado
contacto_cliente
correo_contacto
telefono_contacto
fecha_promesa_pago
monto_prometido
completada
```

### Migración 3 — Alertas

Agregar:

```
nuevos tipos
gestion_cobranza
clave_control
motivo_resolucion o descarte
```

### Migración 4 — Índices

En `GestionCobranza`:

```
models.Index(fields=["factura", "-fecha"])
models.Index(fields=["proximo_contacto"])
models.Index(fields=["fecha_promesa_pago"])
```

En `AlertaCobranza`:

```
models.Index(fields=["estado", "tipo"])
models.Index(fields=["factura", "estado"])
models.Index(fields=["pago", "estado"])
```

En `AplicacionMovimientoBancario`:

```
models.Index(fields=["pago_cliente", "activa"])
```

## 8. Definición de terminado del bloque

F008-F014 se considerará terminado cuando:

- Los pagos sin aplicar son visibles.
- Los excedentes pueden clasificarse.
- Los saldos pueden regularizarse.
- Los pagos pueden vincularse con uno o varios movimientos.
- No se sobreconcilian pagos ni movimientos.
- Se registran las gestiones de cobranza.
- Se registran compromisos de pago.
- Se generan alertas de vencimiento.
- Se generan alertas por promesas incumplidas.
- Se generan alertas por falta de respuesta.
- Las alertas no se duplican.
- Existe un reporte de antigüedad.
- Los saldos se agrupan por cliente y tramo.
- Factoring se presenta por separado.
- Los reportes pueden exportarse.
- Pantalla y exportación coinciden.
- Las operaciones quedan auditadas.
- Las pruebas integrales pasan.

## 9. Riesgos del bloque

| Riesgo                                           | Impacto | Tratamiento            |
|--------------------------------------------------|---------|------------------------|
| Confundir saldo de factura con saldo de pago     | Alto    | Propiedades separadas  |
| Confundir distribución con conciliación bancaria | Alto    | Estados independientes |
| Sobreconciliar pago                              | Alto    | Bloqueos y validación  |
| Sobreutilizar movimiento bancario                | Alto    | Servicio transaccional |
| Excedente olvidado                               | Alto    | Bandeja y alertas      |

| Riesgo                                | Impacto        | Tratamiento                         |
|---------------------------------------|----------------|-------------------------------------|
| Alertas duplicadas                    | Medio/<br>alto | Clave de control                    |
| Promesa sin seguimiento               | Alto           | Fecha estructurada                  |
| Aging incorrecto                      | Alto           | Fecha de corte y pruebas de límites |
| Factoring mezclado con cartera propia | Alto           | Secciones separadas                 |
| Reportes distintos a pantalla         | Alto           | Selectores reutilizados             |
| Exportación de datos no autorizados   | Alto           | Permisos y auditoría                |
| Concurrencia                          | Alto           | <code>select_for_update()</code>    |
| Anulaciones sin trazabilidad          | Alto           | Estados e historial                 |

## 10. Resultado consolidado

Al finalizar este bloque se completa el ciclo:

Factura  
 → pago  
 → aplicación  
 → saldo o excedente  
 → conciliación bancaria  
 → cobranza  
 → alerta  
 → antigüedad  
 → reporte  
 → pruebas

El módulo de facturación quedará preparado para integrarse con:

B010-B014  
 Conciliación bancaria avanzada

X001-X013  
 Factoring

Q001-Q002  
 Pruebas end-to-end y aceptación